



Lab 04 – Worksheet

Name: M. Shees Abbas	ID: ma11212	Section: T4
----------------------	-------------	-------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Provide appropriately commented codes.

Part (a)

```
.text
.globl main

main:
    li    x10, 3
    jal   x1, fact    # call fact(3)
    j     end         # stop after return

fact:
    addi  sp, sp, -8   #adjust stack for 2 items
    sw    x1, 4(sp)    #save return address
    sw    x10, 0(sp)   #save argument n

    addi  x5, x10, -1  #x5 = n - 1
    bge   x5, x0, L1   #if (n - 1) >= 0, go to L1

    addi  x10, x0, 1   #return 1
    addi  sp, sp, 8    #pop stack
    jalr  x0, 0(x1)

L1:
    addi  x10, x10, -1 #argument = n - 1
    jal   x1, fact     #recursive call

    addi  x6, x10, 0    #save result of fact(n-1)
    lw    x10, 0(sp)    #restore original n
    lw    x1, 4(sp)     #restore return address
    addi  sp, sp, 8     #pop stack
```



```
mul    x10 , x10 , x6 #n * fact(n-1)
jalr   x0 , 0(x1)     #return

end:
j      end
```

Part (b)

```
.text
.globl main

main:
    li    x10, 3
    jal   x1, fact    # call fact(3)
    j     end         # stop after return

fact:
    addi   x5 , x0 , 1

L1:
    bge    x0 , x10 , exit #if (0 >= n), exit loop
    mul    x5 , x5 , x10
    addi   x10 , x10 , -1  #n = n - 1
    jal    x0 , L1        #repeat

exit:
    addi   x10 , x5 , 0    #return acc (move result into x10)
    jalr   x0 , 0(x1)     #return

end:
j      end
```



**Add snip of relevant Registers / Memory locations where you think results are obtained. Make sure the irrelevant area of snip is cropped. Make sure the irrelevant area of snip is cropped.*

Part (a)

PC before and after function call

PC = 0x00000004 PC = 0x0000000C

PC before and after calling L1

PC = 0x0000001C PC = 0x0000002C

PC before and after coming back to function recursively

PC = 0x00000030 PC = 0x0000000C

Memory after final recursion

0x7FFFFFF0	00	00	00	00
0x7FFFFFFC	08	00	00	00
0x7FFFFFF8	03	00	00	00
0x7FFFFFF4	34	00	00	00
0x7FFFFFF0	02	00	00	00
0x7FFFFDC	34	00	00	00
0x7FFFFD8	01	00	00	00
0x7FFFFD4	34	00	00	00
0x7FFFFD0	00	00	00	00

Final register values after considering input as 3 the x10 contains the output

x00 (zero) = 0x00000000
x01 (ra) = 0x00000008
x02 (sp) = 0x7FFFFFF0
x03 (gp) = 0x10000000
x04 (tp) = 0x00000000
x05 (t0) = 0xFFFFFFFF
x06 (t1) = 0x00000002
x07 (t2) = 0x00000000
x08 (s0) = 0x00000000
x09 (s1) = 0x00000000
x10 (a0) = 0x00000006
x11 (a1) = 0x00000000
x12 (a2) = 0x00000000
x13 (a3) = 0x00000000
x14 (a4) = 0x00000000
x15 (a5) = 0x00000000



Part b

PC before and after fact function call:

PC = 0x00000004 PC = 0x0000000C

PC before and after while loop implementation:

PC = 0x0000001C PC = 0x00000010

PC before and after exiting the loop

PC = 0x00000010 PC = 0x00000020

PC before and after returning from function

PC = 0x00000024 PC = 0x00000008

Stack:

There was no need to implement any stack as this function never call another function or itself, as it was a leaf function

Final register values after considering the input as 3, x10 register contains the results

```
x00 (zero) = 0x00000000
x01 (ra)    = 0x00000008
x02 (sp)    = 0x7FFFFFF0
x03 (gp)    = 0x10000000
x04 (tp)    = 0x00000000
x05 (t0)    = 0x00000006
x06 (t1)    = 0x00000000
x07 (t2)    = 0x00000000
x08 (s0)    = 0x00000000
x09 (s1)    = 0x00000000
x10 (a0)    = 0x00000006
x11 (a1)    = 0x00000000
x12 (a2)    = 0x00000000
```



Comments

Part (a):

In the part a we used recursion to find the factorial that's why we had to use implement stack which resulted in a greater number of instructions to be executed

Part (b):

In this part we used while loop to find the factorial that's why we didn't had to put our register values in the stack as it wasn't calling any other function or itself

Task 2

Provide appropriately commented codes

```
.text
.globl main

main:
    li    x10, 3
    li    x7, 1
    jal   x1, fact    # call fact(3)
    j     end         # stop after return

fact:
    addi   sp, sp, -8    #adjust stack for 2 items
    sw     x1, 4(sp)     #save return address
    sw     x10, 0(sp)    #save argument n

    addi   x5, x10, -1   #x5 = n - 1
    bge    x5, x7, L1    #if (n - 1) >= 1, go to L1

    addi   x10, x0, 1    #return 0
    addi   sp, sp, 8     #pop stack
    jalr   x0, 0(x1)

L1:
    addi   x10, x10, -1  #argument = n - 1
    jal    x1, fact      #recursive call

    addi   x6, x10, 0    #save result of fact(n-1)
    lw     x10, 0(sp)    #restore original n
    lw     x1, 4(sp)     #restore return address
    addi   sp, sp, 8     #pop stack
```



```
add    x10 , x10 , x6    #n + fact(n-1)
jalr   x0 , 0(x1)        #return

end:
j      end
```

Add screenshot of your results

The stack populated in first go and the sp value:

0x7FFFFFFEC	08	00	00	00
0x7FFFFFFE8	03	00	00	00

x02 (sp) = 0x7FFFFFFE8

Stack populated in second go and the sp value:

0x7FFFFFFEC	08	00	00	00
0x7FFFFFFE8	03	00	00	00
0x7FFFFFFE4	34	00	00	00
0x7FFFFFFE0	02	00	00	00

x02 (sp) = 0x7FFFFFFE0

The stack populated in final go:

0x7FFFFFFEC	0C	00	00	00
0x7FFFFFFE8	03	00	00	00
0x7FFFFFFE4	38	00	00	00
0x7FFFFFFE0	02	00	00	00
0x7FFFFFDC	38	00	00	00
0x7FFFFFD8	01	00	00	00
0x7FFFFFD4	00	00	00	00

x02 (sp) = 0x7FFFFFD8

Final register values after assuming input as 3 and the final result is stored in x10

x00 (zero) = 0x00000000
x01 (ra) = 0x0000000C
x02 (sp) = 0x7FFFFFF0
x03 (gp) = 0x10000000
x04 (tp) = 0x00000000
x05 (t0) = 0x00000000
x06 (t1) = 0x00000003
x07 (t2) = 0x00000001
x08 (s0) = 0x00000000
x09 (s1) = 0x00000000
x10 (a0) = 0x00000006
x11 (a1) = 0x00000000



Comments

I just used the code provided in the task1(a) and just change the base case as it first was checking for if x5 was greater than or equal to 0 but now it check for greater than equal to 1 and then do recursion

Task 3

Provide appropriately commented codes

```
.text
.globl main

main:
    li    x10, 0x100
    li    x11, 4
    li    x5, 3
    sw    x5, 0(x10)
    li    x5, 1
    sw    x5, 4(x10)
    li    x5, 4
    sw    x5, 8(x10)
    li    x5, 2
    sw    x5, 12(x10)
    jal   x1, bubble
    j     end

bubble:
    beq   x10, x0, exit
    beq   x11, x0, exit

    li    x5, 0
loop_i:
    bge   x5, x11, exit
    addi  x6, x5, 0
loop_j:
    bge   x6, x11, next_i

    slli  x7, x5, 2
    add   x7, x10, x7
    lw    x28, 0(x7)
```



```
slli x29, x6, 2
add x29, x10, x29
lw x30, 0(x29)

bge x28, x30, skip
sw x30, 0(x7)
sw x28, 0(x29)

skip:
addi x6, x6, 1
j loop_j

next_i:
addi x5, x5, 1
j loop_i

exit:
jalr x0, 0(x1)

end:
j end
```

Add screenshots of your results

The unsorted array values

0x00000110	00	00	00	00
0x0000010C	02	00	00	00
0x00000108	04	00	00	00
0x00000104	01	00	00	00
0x00000100	03	00	00	00
0x000000FC	00	00	00	00

After first iteration of outer loop:

0x0000010C	02	00	00	00
0x00000108	03	00	00	00
0x00000104	01	00	00	00
0x00000100	04	00	00	00

Final sorted array:

0x0000010C	01	00	00	00
0x00000108	02	00	00	00
0x00000104	03	00	00	00
0x00000100	04	00	00	00

**Final register values**

```
x00 (zero) = 0x00000000
x01 (ra)    = 0x0000002C
x02 (sp)    = 0x7FFFFFF0
x03 (gp)    = 0x10000000
x04 (tp)    = 0x00000000
x05 (t0)    = 0x00000004
x06 (t1)    = 0x00000004
x07 (t2)    = 0x0000010C
x08 (s0)    = 0x00000000
x09 (s1)    = 0x00000000
x10 (a0)    = 0x00000100
x11 (a1)    = 0x00000004
x12 (a2)    = 0x00000000
x13 (a3)    = 0x00000000
x14 (a4)    = 0x00000000
x15 (a5)    = 0x00000000
x16 (a6)    = 0x00000000
```

Comments:

In this code we used two nested loops to bubble sort and to avoid using the stack we used unconditional jumps without saving the return value, in order jump to the next iteration of loops via labels

Task 4

Provide appropriately commented codes

```
.text
.globl main

main:
    li    x10, 0x200
    li    x11, 5

    li    x5, 1                #array being populated
    sw    x5, 0(x10)
    li    x5, 2
```



```
sw    x5, 4(x10)
li    x5, 3
sw    x5, 8(x10)
li    x5, 4
sw    x5, 12(x10)
li    x5, 5
sw    x5, 16(x10)

jal   x1, sumOfSquares    #call the func
j     end

#
sumOfSquares:
    addi sp, sp, -20      #saving value before the sceond function
call
    sw    x1, 16(sp)
    sw    x8, 12(sp)      # as x8, x9, x18 and x19 are saved
register so to use them we gonna store their values
    sw    x9, 8(sp)
    sw    x18, 4(sp)
    sw    x19, 0(sp)

    addi x8, x10, 0        # x8 = a
    addi x9, x11, 0        # x9 = len
    li    x18, 0           # x18 = i = 0
    li    x19, 0           # x19 = sum = 0

loop:
    bge   x18, x9, done    #if (i >= len) exit loop

    slli  x6, x18, 2        #x6 = i * 4
    add   x6, x8, x6        #x6 = address of a[i]
    lw    x10, 0(x6)        #argument for square function

    jal   x1, square        #call square function with argument in
x10
    add   x19, x19, x10     #sum += sq (result returned in x10)
    addi  x18, x18, 1       #i++
    j     loop             #repeat

done:
    addi  x10, x19, 0       #return value = sum

    lw    x1, 16(sp)        #restore return address
    lw    x8, 12(sp)        #restore s0
```



```
lw    x9, 8(sp)           #restore s1
lw    x18, 4(sp)          #restore s2
lw    x19, 0(sp)          #restore s3
addi  sp, sp, 20

jalr  x0, 0(x1)           #return to caller

square:
mul   x10, x10, x10       #return x * x
jalr  x0, 0(x1)           #return to caller

end:
j     end
```

Add screenshots of your results

Saved register before function call:

x08 (s0)	= 0x00000001	x18 (s2)	= 0x00000004
x09 (s1)	= 0x00000005	x19 (s3)	= 0x00000002

Saved registers stored in stack:

0x7FFFFFFC	44	00	00	00
0x7FFFFFF8	01	00	00	00
0x7FFFFFF4	05	00	00	00
0x7FFFFFE0	04	00	00	00
0x7FFFFFDC	02	00	00	00

Saved register during and after function:

x08 (s0)	= 0x00000200	x18 (s2)	= 0x00000001
x09 (s1)	= 0x00000005	x19 (s3)	= 0x00000001

x08 (s0)	= 0x00000001	x18 (s2)	= 0x00000004
x09 (s1)	= 0x00000005	x19 (s3)	= 0x00000002

Final register values:

x00 (zero)	= 0x00000000	x17 (a7)	= 0x00000000
x01 (ra)	= 0x00000044	x18 (s2)	= 0x00000004
x02 (sp)	= 0x7FFFFFF0	x19 (s3)	= 0x00000002
x03 (gp)	= 0x10000000	x20 (s4)	= 0x00000000
x04 (tp)	= 0x00000000	x21 (s5)	= 0x00000000
x05 (t0)	= 0x00000005	x22 (s6)	= 0x00000000
x06 (t1)	= 0x00000210	x23 (s7)	= 0x00000000
x07 (t2)	= 0x00000000	x24 (s8)	= 0x00000000
x08 (s0)	= 0x00000001	x25 (s9)	= 0x00000000
x09 (s1)	= 0x00000005	x26 (s10)	= 0x00000000
x10 (a0)	= 0x00000037	x27 (s11)	= 0x00000000
x11 (a1)	= 0x00000005	x28 (t3)	= 0x00000000
x12 (a2)	= 0x00000000	x29 (t4)	= 0x00000000
x13 (a3)	= 0x00000000	x30 (t5)	= 0x00000000
x14 (a4)	= 0x00000000	x31 (t6)	= 0x00000000
x15 (a5)	= 0x00000000		
x16 (a6)	= 0x00000000		

**Comments:**

This code first populates an array with elements 1,2,3,4,5 and save random values in x8,x9,x10,11 to demonstrate the stack implemented when we use these saved register in the function then in the main function it first save the saved registers values then it iterate ove the array and on each element it call another function square which gives a the square of that element which it then store in register x19 and the at the end it return the total sum from x19 into x10 and then it restores the values of all the saved registers back

Git hub repo link: <https://github.com/M-Shees-Abbas/CA-labs.git>



Lab 04 – Nest Procedures and Sorting Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	AR 7 Report Submission & Github
Task 1	/10	/05	/10 & /10
Task 2	/10	/10	
Task 3	/10	/10	
Task 4	/15	/10	
Total Points	/100 Points		
CLO Mapped	CLO 2		

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.



			Report is submitted after due date.		
--	--	--	--	--	--